# Formula Cheat Sheet

### Linear classification (T1–4)

Decision: $+1$ iff $\mathbf{w}^\top f(\mathbf{x}) > 0$ (bias = constant feature 1). BoW: one dimension per vocab word (count or 0/1). $n$-grams in $L$ tokens: $L-n+1$. tf-idf = count $\times \log \frac{\#\text{docs}}{\#\text{docs with } w}$.

**Perceptron:** on a mistake, $\mathbf{w} \leftarrow \mathbf{w} + \alpha\, y\, f(\mathbf{x})$. Loss $\max(0, -y\mathbf{w}^\top f(\mathbf{x}))$. Converges iff separable. $\alpha$-invariant from $\mathbf{w}_0 = 0$.

**LR:** $P(y{=}{+1}|\mathbf{x}) = \sigma(\mathbf{w}^\top f(\mathbf{x})) = \frac{1}{1+e^{-\mathbf{w}^\top f(\mathbf{x})}}$. NLL $= -\log P(y|\mathbf{x}) = \log(1+e^{-yz})$. $\nabla_\mathbf{w} = -y f(\mathbf{x})(1-P(y|\mathbf{x}))$. Update $\mathbf{w} \leftarrow \mathbf{w} + \alpha y f(\mathbf{x})(1-P(y|\mathbf{x}))$. 0/1 labels: $\mathbf{w} \leftarrow \mathbf{w} + \alpha(y-p)f(\mathbf{x})$. Convex.

### Multiclass (T5–6)

$\hat{y} = \arg\max_y \mathbf{w}_y^\top f(\mathbf{x}) = \arg\max_y \mathbf{w}^\top f(\mathbf{x}, y)$ (DF: $f(\mathbf{x})$ copied into block $y$). **MC perceptron:** on a mistake, $\mathbf{w}_{\hat{y}} \mathrel{-}= \alpha f(\mathbf{x})$, $\mathbf{w}_{y^*} \mathrel{+}= \alpha f(\mathbf{x})$. **Softmax/MC LR:** $P(y|\mathbf{x}) = \frac{e^{\mathbf{w}_y^\top f(\mathbf{x})}}{\sum_{y'} e^{\mathbf{w}_{y'}^\top f(\mathbf{x})}}$; $\nabla_{\mathbf{w}_y} = (P(y|\mathbf{x}) - \mathbb{1}[y{=}y^*])f(\mathbf{x})$. Update: gold $+\alpha(1-P)f(\mathbf{x})$, others $-\alpha P(y) f(\mathbf{x})$.

### Optimization (T7)

SGD: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla \mathcal{L}_i$. On $a(w-c)^2$: error $\times(1-2a\alpha)$ per step; converge iff $0 < \alpha < 1/a$. $w^2$: $\alpha = 1$ oscillates, $\alpha = \frac{1}{2}$ one step. A2: $(x_1{-}1)^2{+}8(x_2{-}1)^2$ needs $\alpha < 1/8$. Newton $\mathbf{w} - (\nabla^2\mathcal{L})^{-1}\nabla\mathcal{L}$ (quadratic cost). Adam = adaptive step + momentum. **Init:** zeros kill hidden layers; too large saturates. Xavier $U[\pm\sqrt{6/(\text{in}+\text{out})}]$.

### FFNN & training (T8–9)

$\mathbf{z} = g(Vf(\mathbf{x}))$, $P = \text{softmax}(W\mathbf{z})$; $V{:}\, d \times n$, $W{:}\, C \times d$. Params $dn + Cd$ ($+d{+}C$ biases). Backprop: $\partial\mathcal{L}/\partial W = (\mathbf{p} - \mathbf{e}^*)\mathbf{z}^\top$; $\delta_z = W^\top(\mathbf{p} - \mathbf{e}^*)$; $\partial\mathcal{L}/\partial V = (\delta_z \odot g'(\mathbf{a}))f(\mathbf{x})^\top$. $\tanh' = 1 - \tanh^2$, $\sigma' = \sigma(1-\sigma)$. Loop: zero_grad, forward, loss, backward, step. NLLLoss takes log-probs; CrossEntropyLoss takes logits.

### Embeddings & DAN (T10–11)

Skip-gram: $P(c{=}y|w{=}x) = \frac{e^{\mathbf{v}_x \cdot \mathbf{c}_y}}{\sum_{y' \in V} e^{\mathbf{v}_x \cdot \mathbf{c}_{y'}}}$; params $2|V|d$; one-pair cost $O(|V|d)$. Pairs per sentence: $2(kL - k(k{+}1)/2)$. CBOW: sum of contexts predicts the word. SGNS: $\log\sigma(\mathbf{v}\cdot\mathbf{c}) + \sum_k \log\sigma(-\mathbf{v}\cdot\mathbf{c}')$. cos $= \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$. Debias: $\mathbf{w}' = \mathbf{w} - \text{proj}_{\text{gender}}\mathbf{w}$. DAN: $\text{softmax}(Wg(V\frac{1}{n}\sum e(w_i)))$, order-blind.

### $n$-gram LMs & perplexity (T12, T15)

$P(w|\text{ctx}) = \frac{c(\text{ctx},w)}{c(\text{ctx})}$; bigram: $\prod P(w_i|w_{i-1})$ starting from $\langle s\rangle$. Abs. discount: $\frac{\max(c-k,0)}{c(\text{ctx})} + \lambda P(w|\text{shorter})$, $\lambda = \frac{k\cdot\#\text{types}}{c(\text{ctx})}$. PPL $= \exp(-\frac{1}{N}\sum \ln P(w_i|w_{<i})) = (\prod p_i)^{-1/N}$; uniform gives $|V|$; lower is better.

### Self-attention (T13)

$Q = XW^Q$, $K = XW^K$, $V = XW^V$; $A = \text{softmax}_{\text{row}}(QK^\top/\sqrt{d_k})$ ($n \times n$); out $= AV$. Cost $O(nd\,d_k + n^2 d_k)$. MHSA: $\text{Concat}(\text{head}_1..\text{head}_h)W^O$, params $4d^2$ when $hd_k = d$. Causal mask: $-\infty$ above the diagonal before softmax. Positions: learned absolute (added), sinusoidal, relative (T5), ALiBi ($-m\cdot$dist), NoPE.

### Transformer (T14–15)

$\mathbf{H} = \text{LN}(X + \text{MHSA}(X))$; out $= \text{LN}(\mathbf{H} + \text{FFN}(\mathbf{H}))$; FFN $= W_2\text{ReLU}(W_1\mathbf{h})$, position-wise. Params/layer $4d^2 + 2d\,d_{ff}$ (FFN $\approx 2/3$). Base: $d$=512, $d_{ff}$=2048, $h$=8, $d_k$=64, $N$=6. LM: $P(\cdot|w_{\le i}) = \text{softmax}(W\mathbf{h}_i)$, $W{:}\, |V| \times d$. Input $\langle s\rangle w_1..w_{n-1}$, targets $w_1..w_n$. Output $[B, n, |V|]$, attention $[B, n, n]$. A3: uniform PPL 27; yours 5.63.

### BERT & subwords (T16–17)

MLM: mask 15% (paper: 80% [MASK]/10% random/10% same) + NSP via [CLS] (50/50). Base 12L/768/110M; Large 24L/1024/16 heads/340M. Fine-tune everything, small LR. Encoder = bidirectional; decoder = causal; enc–dec adds cross-attention (BART reconstructs all; T5 predicts spans). BPE: merge the most frequent adjacent pair, repeatedly. WordPiece/unigram LM: pick pieces by likelihood. Greedy = longest match left to right.

### Decoding (T18)

Greedy = argmax each step. Beam $k$: keep the top $k$ partial sequences by product of probabilities; about $kn$ model calls, $k|V|n$ hypotheses. Nucleus $p$: sort, keep the smallest prefix with cumulative $\ge p$, renormalize, sample.

### HMM (T19–20)

$P(\mathbf{y}, \mathbf{x}) = P(y_1)\prod P(y_i|y_{i-1})P(\text{STOP}|y_n)\prod P(x_i|y_i)$. Estimate: count and normalize (emissions $P(\text{word}|\text{tag})$). Viterbi: $v_1(y) = \log P(y) + \log P(x_1|y)$; $v_i(y) = \log P(x_i|y) + \max_{y'}[v_{i-1}(y') + \log P(y|y')]$; end $+\log P(\text{STOP}|y)$. $O(nk^2)$ time; $k^n$ paths; $kl^{n-1}$ if $l$ successors.

### PCFG / CKY (T21–22)

$P(\text{tree}) = \prod P(\text{rule}|\text{parent})$, normalized per parent. Estimate: count and normalize. Binarize (lossless: intermediates have prob 1). Vertical Markov: NP^S, NP^VP (subset language). Horizontal $h$: remember $h$ siblings. CKY: $t(i, i{+}1, X) = \log P(X \to w)$; $t(i,j,X) = \max_{k, X\to YZ}[\log P + t(i,k,Y) + t(k,j,Z)]$; $O(n^3|G|)$.

# Top 30 Traps

1. Perceptron tie rule: lecture $> 0$ (0 is negative); Fall 2023 used $\geq 0$. Use what's stated.
2. A mistake with $f(\mathbf{x}) = \mathbf{0}$ changes nothing.
3. Follow the given vocabulary order; OOV/typo'd words give zeros.
4. tf-idf changes values, not dimensionality; bigrams enlarge the space.
5. LR step: $+\alpha y f(\mathbf{x})(1 - P(y|\mathbf{x}))$; for $y = -1$ use $1 - P(-1) = P(+1)$.
6. The LR boundary is the same as the perceptron's ($\mathbf{w}^\top f(\mathbf{x}) = 0$); it updates on every example.
7. MC perceptron: down-weight the predicted class, up-weight the gold class.
8. MC LR non-gold update is $-\alpha P(y) f(\mathbf{x})$.
9. $w^2$ with $\alpha = 1$ oscillates (not diverges); $\alpha > 1$ diverges.
10. Zero init kills NNs; it's fine for linear models.
11. No nonlinearity means linear; no tanh before softmax.
12. NLLLoss takes log-probs; CrossEntropyLoss takes logits; call zero_grad.
13. Skip-gram denominator: every vocab word, including $x$ itself.
14. $P(y|x)$ uses $\mathbf{v}_x \cdot \mathbf{c}_y$ (word · context).
15. The zero vector gives a uniform distribution (Fall 2023 erratum).
16. Dot product favors long vectors; cosine doesn't.
17. DAN has no attention parameters; averaging is order-blind.
18. $n$-gram: recompute numerators *and* context totals when adding data.
19. Perplexity: exp of the *average* NLL over predicted tokens; lower is better.
20. Attention softmax is row-wise; $A$ is $n \times n$; weights aren't parameters.
21. Causal mask: $-\infty$ *before* softmax.
22. Transformer FFN is position-wise; count MHSA and FFN blocks separately.
23. LM output $[B, n, |V|]$; position $i$ predicts token $i + 1$.
24. BERT: MLM loss only on masked positions; bidirectional; can't generate easily.
25. Greedy tokenization = longest match; it may not use the fewest pieces.
26. Beam keeps the top $k$ overall by product probability.
27. Nucleus: include the crossing token, renormalize by the kept mass.
28. HMM emissions are $P(\text{word}|\text{tag})$; include initial and STOP.
29. PCFG probabilities normalize per parent; the tree product includes lexicon rules.
30. Vertical Markovization gives a *subset* language; CKY splits $i < k < j$.

# Calculator Recipes

**General rules for auto-graded answers.** Keep at least 5 significant digits in intermediate steps and round only at the end, exactly as instructed. If asked for a fraction, give a reduced fraction (`9/500`); if a decimal, the stated number of decimal places. Use natural log unless the question gives a base or a value for $e$. If a question says "assume $e = 3$", use 3 everywhere ($e^0 = 1$, $e^1 = 3$, $e^{-1} = 1/3$).

**Calculator recipe: sigmoid / logistic regression step**

$z = \sum_j w_j f_j \to p = 1/(1+e^{-z}) \to$ loss $-\ln p$ ($y{=}+1$) or $-\ln(1-p)$ ($y{=}-1$) $\to$ factor $s = +\alpha(1-p)$ or $-\alpha p \to w_j \leftarrow w_j + s f_j$. Check: $\sigma(0) = 0.5$, $\sigma(1) = 0.73106$, $\sigma(2) = 0.88080$, $\sigma(-1) = 0.26894$.

**Calculator recipe: softmax**

Subtract the max (optional) $\to$ exponentiate each $\to$ divide by the sum. $(2,1,0) \to (0.6652, 0.2447, 0.0900)$; $(2,0,-1) \to (0.8438, 0.1142, 0.0420)$; equal scores $\to$ uniform. With $e = 3$: $(1,0,0) \to (3,1,1)/5$.

**Calculator recipe: log-likelihood / perplexity**

$\text{PPL} = \exp(-\text{mean} \ln p_i)$ or $(\prod p_i)^{-1/N}$. Powers of two: if $\prod p = 2^{-m}$ over $N$ tokens, $\text{PPL} = 2^{m/N}$. A3 check: average log-prob $-3.296$ gives $e^{3.296} = 27$.

**Calculator recipe: $n$-gram probability**

Write every factor $c(\text{ctx}, w)/c(\text{ctx})$ as a fraction; multiply fractions; reduce. If adding data: update numerators *and* context totals.

**Calculator recipe: attention**

Scores $\mathbf{q} \cdot \mathbf{k}_j$ ($\div\sqrt{d_k}$ if scaled) $\to$ (mask: future $= -\infty$) $\to$ softmax per row $\to$ output $= \sum_j \alpha_j \mathbf{v}_j$.

**Calculator recipe: beam search / nucleus**

Beam: list all $k \times |V|$ extensions with products; sort; keep the top $k$. Nucleus: sort descending; accumulate until $\geq p$; renormalize by the kept sum (e.g. $0.08/0.98$).

**Calculator recipe: Viterbi / CKY in log space**

Add logs instead of multiplying; at each cell take the max over predecessors (Viterbi) or over split points and rules (CKY); write the backpointer next to each cell; finish with STOP (HMM) or read $t(0, n, S)$. Lecture CKY uses $\log_2$ ($\log_2 0.5 = -1$).

**Calculator recipe: parameter counts**

Linear $a \to b$: $ab(+b)$. Embedding table $|V|d$. MHSA $4d^2$. FFN $2d\,d_{ff}$. LayerNorm $2d$. Skip-gram $2|V|d$.

# Shape & Parameter Tables

Conventions: lecture math writes weight matrices as (out $\times$ in); PyTorch `nn.Linear(in, out)` stores `weight` as (out, in); exam options write [in $\times$ out]. $B$ = batch, $n$ = sequence length.

## Models and parameters

| model | parameters (no biases unless stated) | example |
|---|---|---|
| BoW perceptron / LR | $d$ ($\|V\|$ + bias) | $\|V\|$=10k $\rightarrow$ 10,001 |
| Multiclass LR | $\|\mathcal{Y}\| \cdot d$ | 20 classes $\times$ 10k = 200,000 |
| FFNN, 1 hidden | $dn + \|\mathcal{Y}\|d$ (+$d$+$\|\mathcal{Y}\|$) | 300$\rightarrow$200$\rightarrow$2: 60,400 (60,602) |
| Skip-gram | $2\|V\|d$ | 50k $\times$ 300 $\times$ 2 = 30M |
| Bigram skip-gram (F22 Q5) | $\|B\|d + \|V\|d$; sum-of-words version $2\|V\|d$ | |
| DAN | $\|V\|d$ (if trained) + FFNN | 15k$\times$300 + 30,302 = 4,530,302 |
| Bigram LM (count) | $\|V\|^2$ table entries | |
| Self-attention head | $W^Q, W^K$: $d \times d_k$; $W^V$: $d \times d_v$ | $d$=64,$d_k$=16: 3,072 |
| MHSA layer | $3d(hd_k) + hd_v d = 4d^2$ if $hd_k = d$ | $d$=512: 1,048,576 |
| FFN sublayer | $2d\,d_{ff}$ (+$d_{ff} + d$) | 512/2048: 2,097,152 |
| Transformer layer | $4d^2 + 2d\,d_{ff}$ (+4$d$ LN +biases) | 3,145,728 |
| Transformer LM | layers + $\|V\|d$ (embed) + $n_{max}d$ (pos) + $d\|V\|$ (out) | your A3: 24,027 total |
| BERT-Large | 24 layers, $d$=1024, 16 heads | 340M |

## Tensor shapes (A2 DAN, A3 LM)

| stage | DAN (A2) | Transformer LM (A3, batched) |
|---|---|---|
| input indices | $[B, n]$ (PAD = 0) | $[B, n]$ (space = start symbol) |
| embeddings | $[B, n, d]$ | $[B, n, d_{model}]$ (+ positions $[n, d_{model}]$ broadcast) |
| pooling / mixing | mean over real tokens $\rightarrow$ $[B, d]$ | $Q, K$: $[B, n, d_k]$; $V$: $[B, n, d_v]$ |
| attention map | — | $QK^\top$, $A$: $[B, n, n]$ (unbatched $[n, n]$) |
| hidden | $[B, h]$ | layer output $[B, n, d_{model}]$ |
| output | log-probs $[B, C]$ | log-probs $[B, n, \|V\|]$ |
| targets | $[B]$ | $[B, n]$; flatten to $[Bn, \|V\|]$ / $[Bn]$ for NLLLoss |

## Runtimes

| computation | cost |
|---|---|
| skip-gram $P(y\|x)$, full softmax | $O(\|V\|d)$; hierarchical softmax $O(d \log \|V\|)$ |
| training $M$ examples $\times N$ epochs | $O(MNE)$, $E$ = per-example cost |
| self-attention | $O(nd\,d_k + n^2 d_k)$ time, $O(n^2)$ memory |
| RNN encoding | $O(n)$ sequential steps (not parallel) |
| beam search | $\approx kn$ model calls, $k\|V\|n$ hypotheses; exhaustive $\|V\|^n$ |
| Viterbi | $O(nk^2)$; paths $O(k^n)$ |
| CKY | $O(n^3\|G\|)$ |